

Sistema de Gerenciamento para Barbearias: Um Estudo de Caso do DevBarberShop

Curso: Ciência da Computação

Johnny Matheus N. de Medeiro

Nathaniel Nicolas Rissi Soares

Nelson Ramos Rodrigues Junior

Resumo

O projeto DevBarberShop foi desenvolvido para gerenciar as operações de uma barbearia, permitindo o cadastro de clientes, fornecedores, funcionários, serviços, produtos, agendamentos e pagamentos. A iniciativa integra conhecimentos adquiridos nas disciplinas de Banco de Dados II, Engenharia de Software I e Programação II, utilizando ferramentas como PostgreSQL, Spring Boot e Visual Paradigm. A aplicação promove eficiência, segurança e escalabilidade, com uma interface intuitiva e relatórios detalhados, atendendo às necessidades práticas de gestão de barbearias.

Palavras chave: Gestão de Barbearias, Banco de Dados, PostgreSQL, Spring Boot, Engenharia de Software, Sistema de Agendamento.

1 Introdução:

Gerenciar operações em barbearias exige um sistema eficiente e robusto, que integre funções administrativas e operacionais de forma automatizada. O projeto DevBarberShop foi desenvolvido para abordar esses desafios, permitindo maior controle sobre dados de clientes, serviços e produtos. Com base nos conhecimentos de disciplinas específicas do curso de Ciência da Computação, o sistema foi projetado para ser funcional, seguro e escalável. Além disso, o projeto é fundamentado em práticas de modelagem de sistemas e desenvolvimento orientado a objetos.

2 Desenvolvimento

2.1 Descrição do Projeto

O DevBarberShop centraliza as principais operações de uma barbearia em uma plataforma única, possibilitando:

- Cadastro de clientes, fornecedores e funcionários.
- Gerenciamento de produtos e serviços.

- Controle de agendamentos e pagamentos.
- Geração de relatórios e análise de históricos de serviços.

2.2 Levantamento de Requisitos

Funcionais:

- Cadastro de Clientes: O sistema deve permitir o cadastro de clientes com informações como nome, telefone, e-mail, endereço, sexo e data de nascimento.
- Cadastro de Fornecedores: O sistema deve permitir o cadastro de fornecedores com informações como nome, telefone, e-mail e endereço.
- Cadastro de Funcionários: O sistema deve permitir o cadastro de funcionários com informações como nome, telefone, e-mail, cargo e salário.
- Cadastro de Serviços: O sistema deve permitir o cadastro de serviços oferecidos pela barbearia, incluindo nome, descrição e preço.
- Cadastro de Produtos: O sistema deve permitir o cadastro de produtos, incluindo nome, descrição, quantidade em estoque, preço, e o fornecedor responsável.
- Agendamento de Serviços: O sistema deve permitir que os clientes agendem serviços com os funcionários, especificando a data, hora e status do agendamento.
- Registro de Pagamentos: O sistema deve permitir o registro de pagamentos realizados pelos clientes para os serviços agendados, incluindo valor, forma de pagamento e data/hora do pagamento.
- Histórico de Serviços: O sistema deve registrar o histórico de serviços prestados, com detalhes sobre o serviço realizado, o cliente atendido, e o funcionário responsável.
- Relatórios e Consultas: O sistema deve possibilitar a consulta e a geração de relatórios sobre agendamentos, pagamentos e histórico de serviços.

Não Funcionais:

- Desempenho: O sistema deve ser capaz de processar as transações de agendamento e pagamento rapidamente, sem causar lentidão no sistema.
- Segurança: O sistema deve garantir a segurança dos dados, com criptografia das informações sensíveis, como e-mails e números de telefone, além de controle de acesso para diferentes perfis (clientes, funcionários e administradores).
- Escalabilidade: O sistema deve ser capaz de se adaptar ao crescimento da barbearia, permitindo a adição de novos funcionários, clientes, serviços e produtos sem impacto significativo no desempenho.
-
- Compatibilidade: O sistema deve ser compatível com diferentes dispositivos e navegadores, garantindo uma experiência fluida para os usuários.
- Usabilidade: A interface do sistema deve ser simples, intuitiva e fácil de navegar, com feedback claro para o usuário em cada interação.
- Backup e Recuperação de Dados: O sistema deve permitir a criação de backups periódicos dos dados, garantindo que as informações possam ser recuperadas em caso de falha.

- Acessibilidade: O sistema deve ser acessível a pessoas com deficiência, seguindo as diretrizes de acessibilidade web, como W3C WCAG.

Requisitos de Domínio:

◆ Cliente:

- Cada cliente deve ter um código de identificação único, nome, telefone, e-mail, endereço (bairro, rua, cidade), sexo e data de nascimento.
- Os dados do cliente são essenciais para o agendamento de serviços e registro de histórico.

◆ Fornecedor:

- Cada fornecedor deve ter um código único, nome, telefone, e-mail e endereço, facilitando o controle dos produtos e insumos.
- Funcionário:
- Os funcionários devem ter um código único, nome, telefone, e-mail, cargo e salário.
- Funcionários são essenciais para o agendamento e execução de serviços.

◆ Serviço:

- Cada serviço deve ter um código, nome, descrição e preço.
- Os serviços são agendados pelos clientes e registrados no histórico de serviços prestados.

◆ Produto:

- Cada produto deve ter um código único, nome, descrição, quantidade em estoque e preço.
- Produtos são vinculados a fornecedores e são essenciais para a execução de certos serviços.

◆ Agendamento:

- Cada agendamento é identificado por um código e contém a data/hora do serviço, cliente e funcionário responsáveis.
- Os agendamentos representam a reserva de um serviço e devem ser gerenciados de acordo com políticas de cancelamento e alteração.

◆ Pagamento:

- Pagamentos são identificados por um código e contêm informações de valor, data/hora e forma de pagamento, além do cliente e agendamento vinculados.
- O pagamento finaliza a prestação de serviço.

♦ Histórico de Serviços:

- O histórico de serviços contém registros de todos os serviços realizados, com data/hora, serviço prestado, cliente e funcionário envolvidos.
- Este histórico é importante para consultas futuras sobre serviços realizados e pode ser usado para análise do perfil do cliente.

2.3 Prototipagem e Modelagem

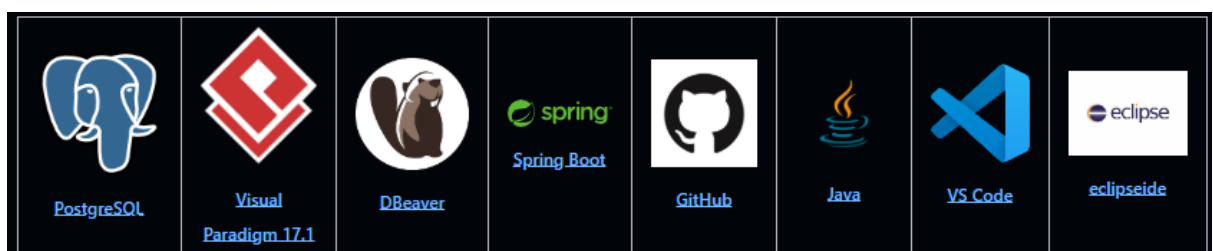
O sistema foi modelado utilizando UML no Visual Paradigm e draw.io, com os seguintes diagramas:

- Diagrama de Casos de Uso: Define as principais interações entre usuários e o sistema.
- Diagrama de Sequência: Representa o fluxo de execução das operações.
- Diagrama de Atividades e Estados: Mapeia os estados do sistema em diferentes cenários.
- Diagrama de Classes: Estrutura as entidades e suas relações no sistema.

2.4 Ferramentas Utilizadas

- PostgreSQL: Banco de dados relacional, com triggers para auditoria e views para relatórios.
- Spring Boot: Desenvolvimento de APIs RESTful e lógica do sistema.
- Java: Linguagem de programação base.
- Eclipse/VS Code: IDEs para codificação e testes.
- Visual Paradigm/ draw.io: Modelagem UML para planejamento do sistema.
- Dbeaver:

Ferramentas utilizadas



fonte: Os autores

2.5 Implementação e Banco de Dados

O banco de dados foi estruturado para garantir integridade e eficiência, contendo as tabelas:

- Cliente
- Fornecedor

- Funcionário
- Serviço
- Produto
- Agendamento
- Pagamento

🎲 Script - Criação do Banco do Dados

--Script - criação do Banco do Dados
create database devbarbershop;

🎲 Script - criação Tabelas

```sql

-- Script de criação de tabelas

```
create table cliente(
 codcli int generated by default as identity primary key,
 nomcli varchar(150) not null,
 telcli varchar(20) not null,
 emailcli varchar(100) not null,
 bairrocli varchar(100) not null,
 ruaendcli varchar(100) not null,
 sexocli varchar(10) not null,
 cidadecli varchar(50) not null,
 dtnascli date not null
);

comment on table cliente is 'Tabela de clientes da Barbearia';
comment on column cliente.codcli is 'Código do cliente';
comment on column cliente.nomcli is 'Nome do cliente';
comment on column cliente.telcli is 'Telefone do cliente';
comment on column cliente.emailcli is 'E-mail do cliente';
comment on column cliente.bairrocli is 'Bairro do cliente';
comment on column cliente.ruaendcli is 'Rua do endereço do cliente';
comment on column cliente.sexocli is 'Sexo do cliente';
comment on column cliente.cidadecli is 'Cidade do cliente';
comment on column cliente.dtnascli is 'Data de nascimento do cliente';
```

-----  
create table fornecedor(  
 codform int generated by default as identity primary key,

```
nomeform varchar(150) not null,
telform varchar(20),
emailform varchar(100),
enderecoform varchar(100)
);

comment on table fornecedor is 'Tabela de fornecedores';
comment on column fornecedor.codform is 'Código do fornecedor';
comment on column fornecedor.nomeform is 'Nome do fornecedor';
comment on column fornecedor.telform is 'Telefone do fornecedor';
comment on column fornecedor.emailform is 'E-mail do fornecedor';
comment on column fornecedor.enderecoform is 'Endereço do fornecedor';
```

```

create table funcionario(
 codfun int generated by default as identity primary key,
 nomefun varchar(100) not null,
 telfun varchar(20) not null,
 emailfun varchar(100) not null,
 cargofun varchar(100) not null,
 salariofun numeric(10, 2) not null
);

comment on table funcionario is 'Tabela de funcionários da Barbearia';
comment on column funcionario.codfun is 'Código do funcionário';
comment on column funcionario.nomefun is 'Nome do funcionário';
comment on column funcionario.telfun is 'Telefone do funcionário';
comment on column funcionario.emailfun is 'E-mail do funcionário';
comment on column funcionario.cargofun is 'Cargo do funcionário';
comment on column funcionario.salariofun is 'Salário do funcionário';
```

```

create table servico(
 codserv int generated by default as identity primary key,
 nomeserv varchar(100) not null,
 descserv varchar(100),
 precoserv numeric(10, 2) not null
);

comment on table servico is 'Tabela de serviços oferecidos pela Barbearia';
comment on column servico.codserv is 'Código do Serviço';
```

```
comment on column servico.nomeserv is 'Nome do serviço';
comment on column servico.descserv is 'Descrição do serviço oferecido pela Barbearia';
comment on column servico.precoserv is 'Preço dos serviços oferecidos pela Barbearia';
```

```

create table produto(
 codprod int generated by default as identity primary key,
 nomeprod varchar(100) not null,
 descprod varchar(255),
 qtdprod int not null,
 precoprod numeric(10, 2) not null,
 fornecedorcodforn int references fornecedor(codforn),
 servicocodserv int references servico(codserv)
);
```

```
comment on table produto is 'Tabela de produtos';
comment on column produto.codprod is 'Código do produto';
comment on column produto.nomeprod is 'Nome do produto';
comment on column produto.descprod is 'Descrição do produto';
comment on column produto.qtdprod is 'Quantidade do produto em estoque';
comment on column produto.precoprod is 'Preço do produto';
comment on column produto.fornecedorcodforn is 'Código do fornecedor do produto';
comment on column produto.servicocodserv is 'Código do serviço relacionado ao produto';
```

```

create table agendamento(
 codagen int generated by default as identity primary key,
 data_hora timestamp not null,
 status varchar(50) not null,
 clientecodcli int references cliente(codcli),
 funcionariocodfun int references funcionario(codfun)
);
```

```
comment on table agendamento is 'Tabela de agendamentos';
comment on column agendamento.codagen is 'Código do agendamento';
comment on column agendamento.data_hora is 'Data e hora do agendamento';
comment on column agendamento.status is 'Status do agendamento';
comment on column agendamento.clientecodcli is 'Código do cliente';
comment on column agendamento.funcionariocodfun is 'Código do funcionário';
```

```

```

```

create table pagamento(
 codpag int generated by default as identity primary key,
 valor numeric(10, 2) not null,
 data_hora_pagamento timestamp not null,
 forma_pagamento varchar(50) not null,
 clientecodcli int references cliente(codcli),
 agendamentocodagen int references agendamento(codagen)
);

comment on table pagamento is 'Tabela de pagamentos';
comment on column pagamento.codpag is 'Código do pagamento';
comment on column pagamento.valor is 'Valor do pagamento';
comment on column pagamento.data_hora_pagamento is 'Data e hora do pagamento';
comment on column pagamento.forma_pagamento is 'Forma de pagamento';
comment on column pagamento.clientecodcli is 'Código do cliente';
comment on column pagamento.agendamentocodagen is 'Código do agendamento';

create table historico_servico(
 codhistorico int generated by default as identity primary key,
 data_hora timestamp not null,
 servicocodserv int references servico(codserv),
 clientecodcli int references cliente(codcli),
 funcionariocodfun int references funcionario(codfun)
);

comment on table historico_servico is 'Tabela de histórico de serviços prestados';
comment on column historico_servico.codhistorico is 'Código do histórico de serviço';
comment on column historico_servico.data_hora is 'Data e hora do serviço';
comment on column historico_servico.servicocodserv is 'Código do serviço';
comment on column historico_servico.clientecodcli is 'Código do cliente';
comment on column historico_servico.funcionariocodfun is 'Código do funcionário';

```

fonte: Os autores

### Destaques técnicos:

- Triggers: Validação e auditoria de operações.
- Views e Consultas Complexas: Relatórios detalhados e otimizações.

- Políticas de Backup: Segurança e recuperação de dados.

## ## 🎲 SQL para Relatórios (com Views e Joins)

'''

-- View para visualizar agendamentos com informações do cliente e funcionário

**create view** view\_agendamentos\_completos **as**

**select** ag.codagen, ag.data\_hora, ag.status, cl.nomcli, cl.telcli, fn.nomefun

**from** agendamento ag

**join** cliente cl **on** ag.clientecodcli = cl.codcli

**join** funcionario fn **on** ag.funcionariocodfun = fn.codfun;

-- View para visualizar pagamentos com detalhes do cliente e agendamento

**create view** view\_pagamentos **as**

**select** pg.codpag, pg.valor, pg.data\_hora\_pagamento, pg.forma\_pagamento, cl.nomcli,

ag.data\_hora

**from** pagamento pg

**join** cliente cl **on** pg.clientecodcli = cl.codcli

**join** agendamento ag **on** pg.agendamentocodagen = ag.codagen;

'''

## ## 🎲 Scripts de Criação do Banco de Dados e Índices

'''

-- Índices para otimização

**create index** idx\_cliente\_nome **on** cliente(nomcli);

**create index** idx\_fornecedor\_nome **on** fornecedor(nomeforn);

**create index** idx\_funcionario\_nome **on** funcionario(nomefun);

**create index** idx\_servico\_nome **on** servico(nomeserv);

'''

## ## 🎲 Políticas de Acesso

'''

-- Criação de usuários e grupos

**create role** gerente **with** login password 'senha\_gerente';

**create role** atendente **with** login password 'senha\_atendente';

-- Privilégios para gerente

**grant** all privileges on all tables in schema public to gerente;

**grant** select, insert, update, delete on all tables in schema public to atendente;

## ## 🎲 Gatilhos (Triggers)

-- Trigger para garantir que a quantidade de produtos nunca seja negativa

```
create or replace function verificar_quantidade_produto()
```

```
returns trigger as $$
```

```
begin
```

```
 if new.qtdprod < 0 then
```

```
 raise exception 'A quantidade de produto não pode ser negativa.';
```

```
 end if;
```

```
 return new;
```

```
end;
```

```
$$ language plpgsql;
```

```
create trigger trg_verificar_quantidade_produto
```

```
before insert or update on produto
```

```
for each row
```

```
execute function verificar_quantidade_produto();
```

-- Trigger de auditoria de agendamentos

```
create or replace function auditoria_agendamento()
```

```
returns trigger as $$
```

```
begin
```

```
 insert into historico_servico (data_hora, servicocodserv, clientecodcli,
funcionariocodfun)
```

```
 values (now(), new.servicocodserv, new.clientecodcli, new.funcionariocodfun);
```

```
 return new;
```

```
end;
```

```
$$ language plpgsql;
```

```
create trigger trg_auditoria_agendamento
```

```
after insert on agendamento
```

```
for each row
```

```
execute function auditoria_agendamento();
```

## ## 🎲 Políticas e Configuração de Backup e Restore

--Backup

```
pg_dump -U nome_usuario -F c -b -v -f /caminho/do/backup/devbarbershop.bak
```

```
devbarbershop
```

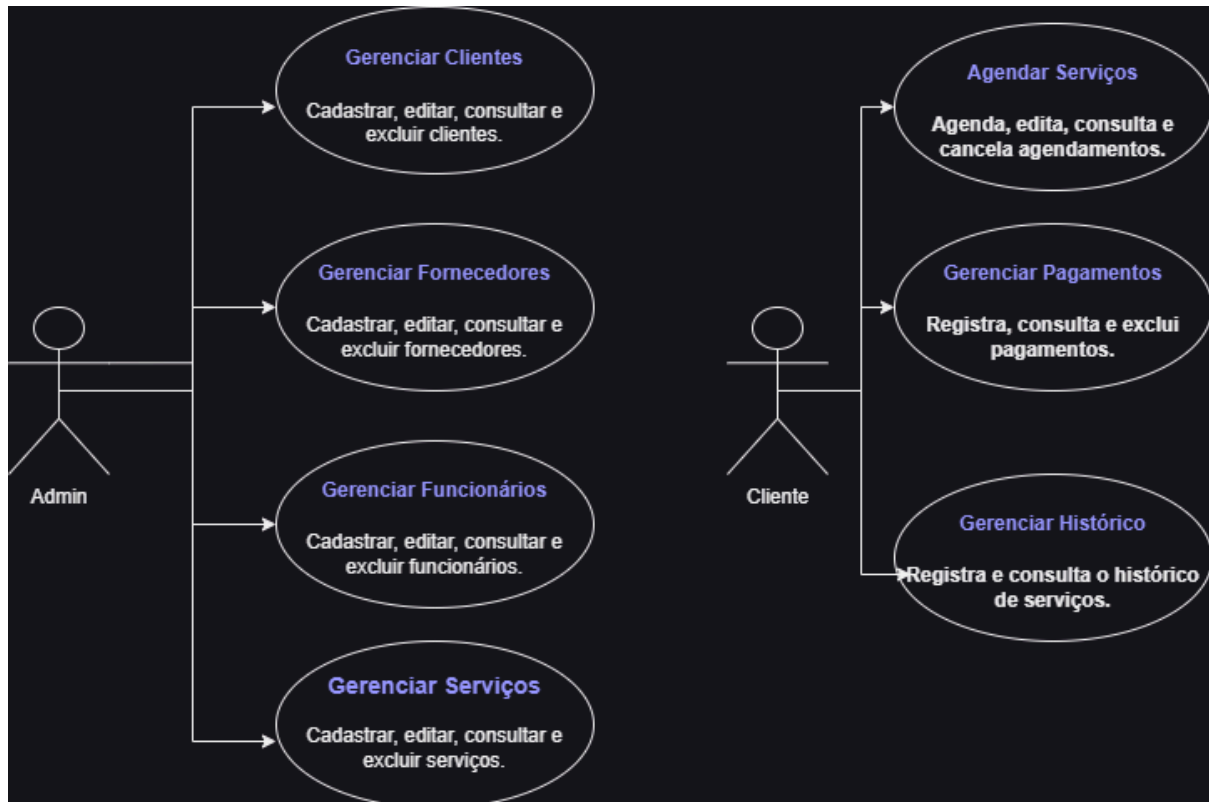
--Restaurar Backup

```
pg_restore -U nome_usuario -d devbarbershop -v /caminho/do/backup/devbarbershop.bak
```

### 3. Diagrama de Caso de Uso

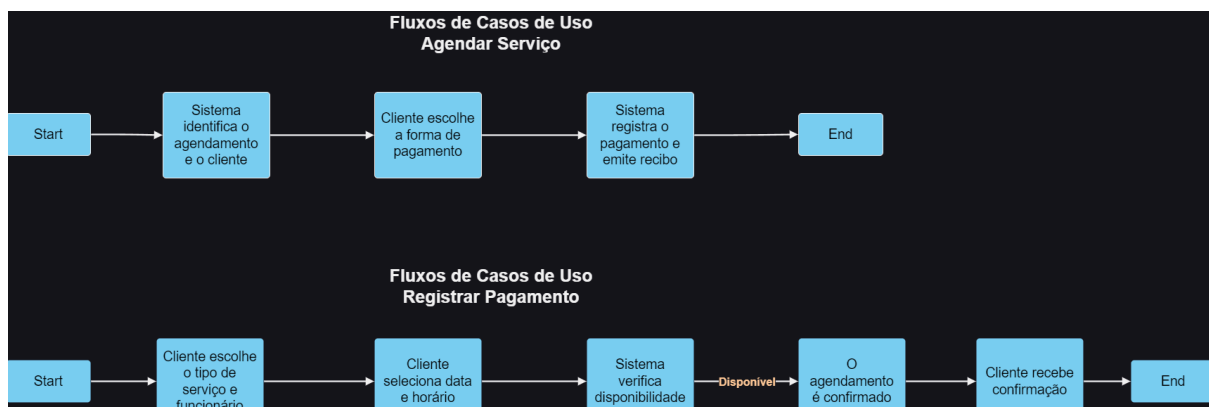
O diagrama ilustra as principais interações do sistema:

- Cliente: Realiza agendamentos e pagamentos.
- Funcionário: Executa serviços.
- Administrador: Gerencia cadastros e relatórios.



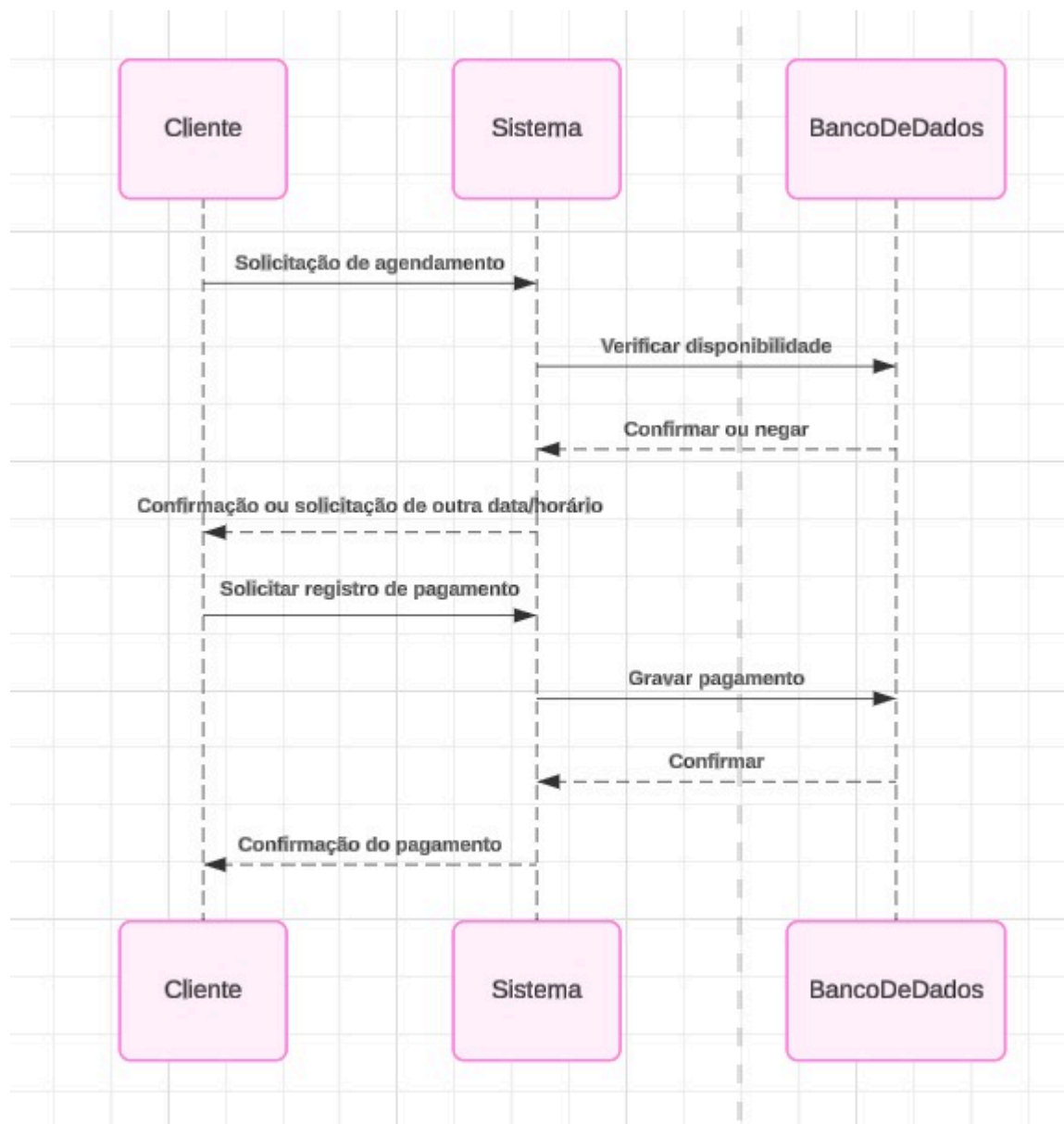
fonte: Os autores

#### 3.1 Diagrama de fluxos



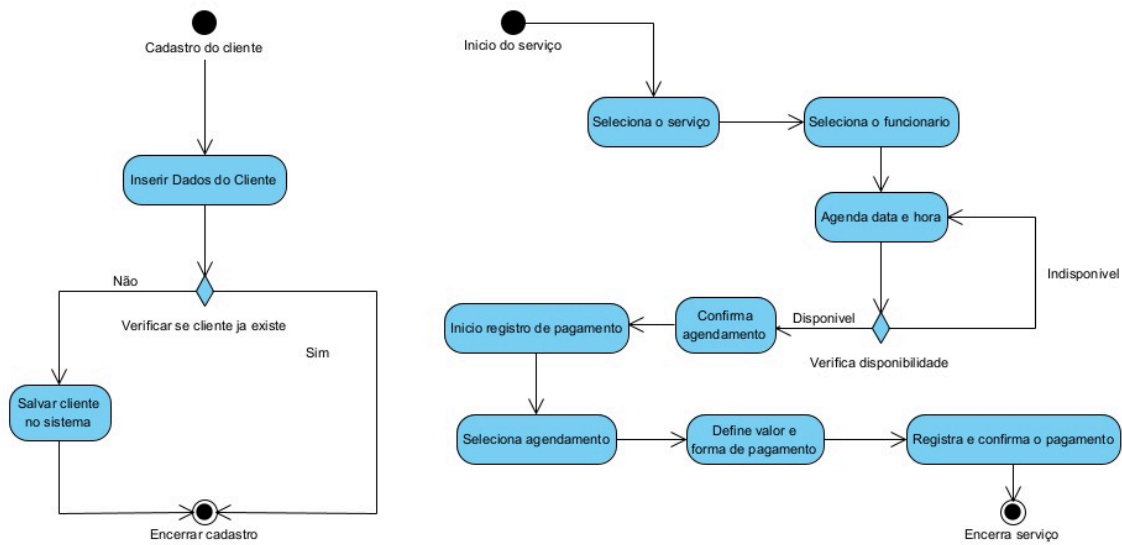
fonte: Os autores

### 3.2 Diagrama de sequência



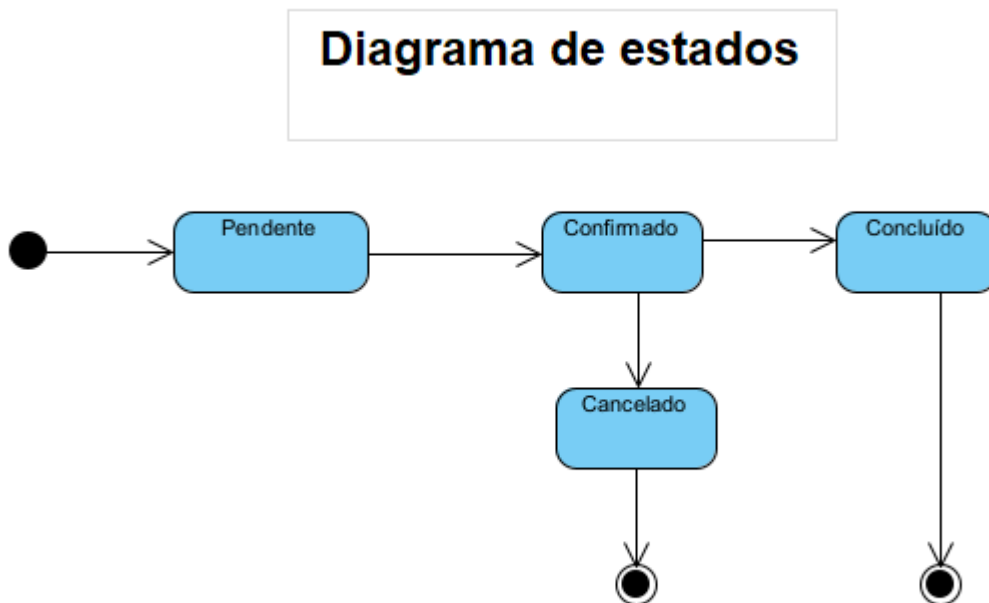
fonte: Os autores

### 3.3 Diagrama de atividades



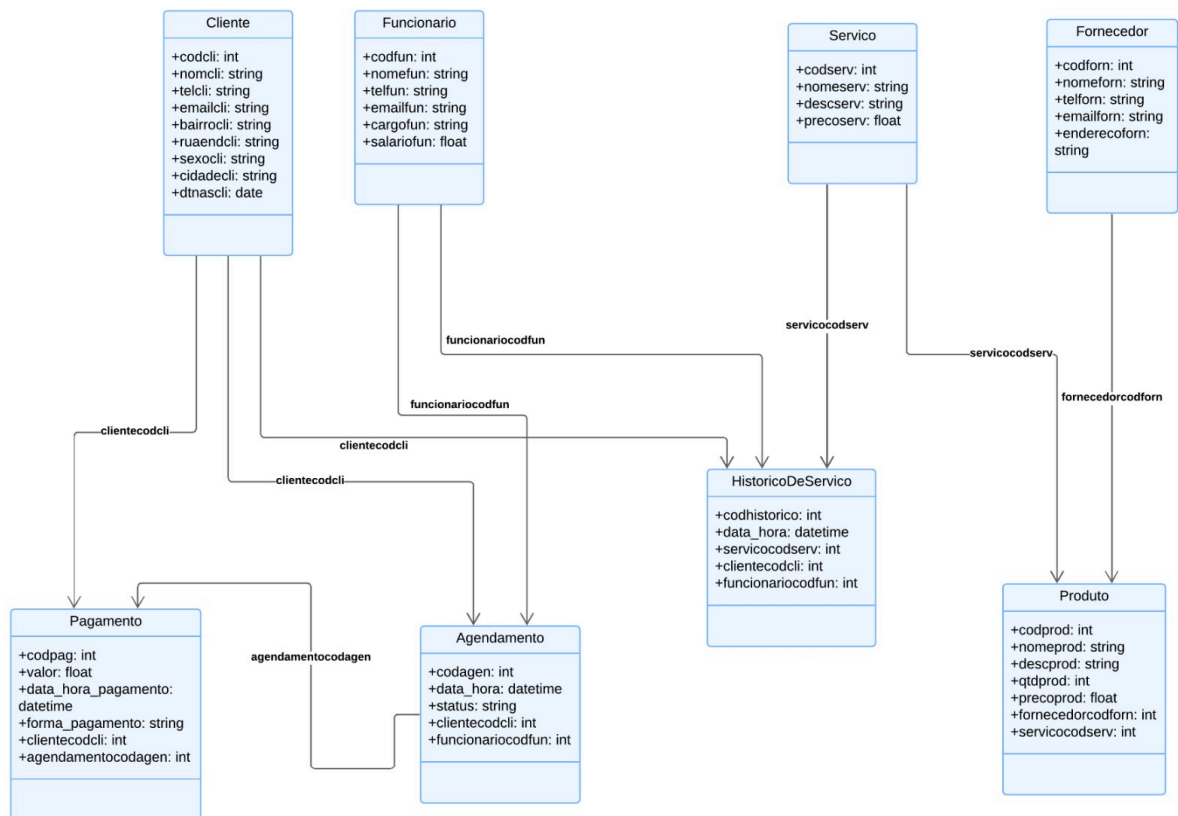
fonte: Os autores

### 3.3 Diagrama de estado



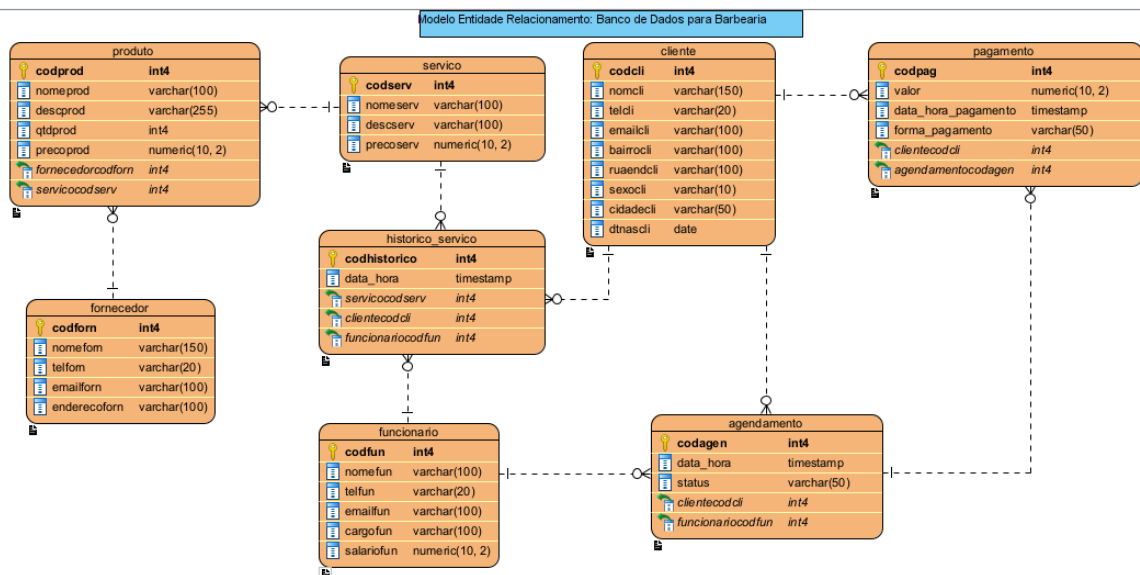
fonte: Os autores

### 3.4 Diagrama de classes



fonte: Os autores

#### 4. Modelo Entidade relacionamento



fonte: Os autores

## 5. Estrutura do projeto

```
src/main/java/com/barbearia
├── controller/
│ ├── ClienteController.java
│ ├── FornecedorController.java
│ ├── FuncionarioController.java
│ ├── ServicoController.java
│ ├── ProdutoController.java
│ ├── AgendamentoController.java
│ └── PagamentoController.java
├── model/
│ ├── Cliente.java
│ ├── Fornecedor.java
│ ├── Funcionario.java
│ ├── Servico.java
│ ├── Produto.java
│ ├── Agendamento.java
│ ├── Pagamento.java
│ └── HistoricoServico.java
└── repository/
 ├── ClienteRepository.java
 ├── FornecedorRepository.java
 ├── FuncionarioRepository.java
 ├── ServicoRepository.java
 ├── ProdutoRepository.java
 ├── AgendamentoRepository.java
 ├── PagamentoRepository.java
 └── HistoricoServicoRepository.java
```

fonte: Os autores

## 6 Implementação do Sistema DevBarberShop com exemplo Cliente

A implementação do sistema DevBarberShop segue o padrão arquitetural Model-View-Controller (MVC), que separa as responsabilidades do sistema em diferentes camadas, promovendo a organização do código e a reutilização de componentes.

### Model:

A camada de Model é responsável por representar as entidades do sistema e gerenciar as interações com o banco de dados. Cada entidade do sistema está mapeada como uma classe Java com o uso de anotações da biblioteca JPA (Java Persistence API).

```
package com.barbearia.BarbeariaApp.model;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "cliente")
public class Cliente {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long codcli;

 private String nomcli;
 private String telcli;
 private String emailcli;
 private String bairrocli;
 private String ruaendcli;
 private String sexocli;
 private String cidadecli;
 private LocalDate dtnascli;
 public Long getCodcli() {
 return codcli;
 }
 public void setCodcli(Long codcli) {
 this.codcli = codcli;
 }
 public String getNomcli() {
 return nomcli;
 }
 public void setNomcli(String nomcli) {
 this.nomcli = nomcli;
 }
 public String getTelcli() {
 return telcli;
 }
 public void setTelcli(String telcli) {
 this.telcli = telcli;
 }
}
```

```
public String getEmailcli() {
 return emailcli;
}
public void setEmailcli(String emailcli) {
 this.emailcli = emailcli;
}
public String getBairrocli() {
 return bairrocli;
}
public void setBairrocli(String bairrocli) {
 this.bairrocli = bairrocli;
}
public String getRuaendcli() {
 return ruaendcli;
}
public void setRuaendcli(String ruaendcli) {
 this.ruaendcli = ruaendcli;
}
public String getSexocli() {
 return sexocli;
}
public void setSexocli(String sexocli) {
 this.sexocli = sexocli;
}
public String getCidadecli() {
 return cidadecli;
}
public void setCidadecli(String cidadecli) {
 this.cidadecli = cidadecli;
}
public LocalDate getDtnascli() {
 return dtnascli;
}
public void setDtnascli(LocalDate dtnascli) {
 this.dtnascli = dtnascli;
}

// Getters e Setters

}
```

## Repository:

A camada de Repository fornece uma interface para comunicação com o banco de dados, utilizando o Spring Data JPA para facilitar as operações de CRUD (Create, Read, Update, Delete).

Exemplo: Repositório ClienteRepository

```
package com.barbearia.BarbeariaApp.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.barbearia.BarbeariaApp.model.Cliente;

public interface ClienteRepository extends JpaRepository<Cliente,
Long> {
}
```

fonte: Os autores

## Controller

A camada de Controller é responsável por gerenciar as solicitações HTTP e delegar as operações necessárias às camadas de negócio e repositório. É implementada com o uso das anotações do Spring Framework.

Exemplo: Controller ClienteController

```
package com.barbearia.BarbeariaApp.controller;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```

import com.barbearia.BarbeariaApp.model.Cliente;
import com.barbearia.BarbeariaApp.repository.ClienteRepository;

@RestController
@RequestMapping("/clientes")
public class ClienteController {
 @Autowired
 private ClienteRepository clienteRepository;

 @GetMapping
 public List<Cliente> listarClientes() {
 return clienteRepository.findAll();
 }

 @GetMapping("/{id}")
 public ResponseEntity<Cliente>
 buscarClientePorId(@PathVariable Long id) {
 Optional<Cliente> cliente =
 clienteRepository.findById(id);
 return cliente.map(ResponseEntity::ok).orElseGet(() ->
 ResponseEntity.notFound().build());
 }

 @PostMapping
 public ResponseEntity<Cliente> adicionarCliente(@RequestBody
 Cliente cliente) {
 Cliente novoCliente = clienteRepository.save(cliente);
 return
 ResponseEntity.status(HttpStatus.CREATED).body(novoCliente);
 }

 @PutMapping("/{id}")
 public ResponseEntity<Cliente> atualizarCliente(@PathVariable
 Long id, @RequestBody Cliente clienteAtualizado) {
 return clienteRepository.findById(id)
 .map(cliente -> {
 cliente.setNomcli(clienteAtualizado.getNomcli());
 cliente.setTelcli(clienteAtualizado.getTelcli());

 cliente.setEmailcli(clienteAtualizado.getEmailcli());

 cliente.setDtnascli(clienteAtualizado.getDtnascli());
 })
 .orElseGet(() -> ResponseEntity.notFound().build());
 }
}

```

```

cliente.setSexocli(clienteAtualizado.getSexocli());

cliente.setBairrocli(clienteAtualizado.getBairrocli());

cliente.setCidadecli(clienteAtualizado.getCidadecli());

cliente.setRuaendcli(clienteAtualizado.getRuaendcli());
 Cliente clienteSalvo =
clienteRepository.save(cliente);
 return ResponseEntity.ok(clienteSalvo);
 })
 .orElseGet(() -> ResponseEntity.notFound().build());
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deletarCliente(@PathVariable Long
id) {
 if (clienteRepository.existsById(id)) {
 clienteRepository.deleteById(id);
 return ResponseEntity.noContent().build();
 }
 return ResponseEntity.notFound().build();
}
}

```

fonte: Os autores

## 7 Conclusão

O DevBarberShop demonstrou ser uma solução eficaz para os desafios encontrados na gestão de barbearias, oferecendo uma plataforma robusta, escalável e intuitiva. A implementação de funcionalidades como cadastro de clientes e fornecedores, controle de estoque, gerenciamento de agendamentos e geração de relatórios detalhados permitiu otimizar os processos operacionais e administrativos. O uso de tecnologias como Spring Boot, PostgreSQL e modelagem UML garantiu uma aplicação moderna e eficiente. Com base nos resultados obtidos, o sistema não apenas atende às demandas do mercado, mas também contribui para o aprimoramento das práticas de desenvolvimento e gestão aprendidas durante o curso, sendo uma base sólida para expansões futuras.

## ***Barbershop Management System: A DevBarberShop Case Study***

### **Abstract**

*The DevBarberShop project was developed to manage the operations of a barbershop, enabling the registration of clients, suppliers, employees, services, products, appointments, and payments. This initiative integrates knowledge acquired from the courses Database II, Software Engineering I, and Programming II, utilizing tools such as PostgreSQL, Spring Boot, and Visual Paradigm. The application promotes efficiency, security, and scalability, featuring an intuitive interface and detailed reports to meet the practical management needs of barbershops.*

**Keywords:** *Barbershop Management, Database, PostgreSQL, Spring Boot, Software Engineering, Appointment System.*

### **Referências**

ORACLE. *Java at a Glance*. Disponível em:

<https://www.oracle.com/java/technologies/java-se-glance.html>. Acesso em: 25 nov. 2024.

DBEAVER. *DBeaver Community Edition Downloads*. Disponível em:

<https://dbeaver.io/download/>. Acesso em: 25 nov. 2024.

SPRING. *Spring Initializr*. Disponível em: <https://start.spring.io/>. Acesso em: 25 nov. 2024.

VISUAL PARADIGM. *Community Edition Downloads*. Disponível em:

<https://www.visual-paradigm.com/download/community.jsp>. Acesso em: 25 nov. 2024.

MICROSOFT. *Visual Studio Code*. Disponível em: <https://code.visualstudio.com/>. Acesso em: 25 nov. 2024.

SPRING Framework Documentation. Disponível em: <https://spring.io/projects/spring-boot>. Acesso em: 25 nov. 2024. Descrição: Documentação oficial do Spring Framework, incluindo práticas recomendadas para construção de APIs RESTful e uso do Spring Data JPA.

BAELDUNG: Spring Boot Guide. Disponível em: <https://www.baeldung.com/spring-boot>.

Acesso em: 25 nov. 2024. Descrição: Guia completo sobre Spring Boot, incluindo explicações detalhadas sobre criação de controladores, repositórios e mapeamento de entidades.

ORACLE. Java SE Technologies. Disponível em: <https://www.oracle.com/java/technologies/>.

Acesso em: 25 nov. 2024. Descrição: Documentação oficial da linguagem Java, com suporte para padrões de design e orientação a objetos utilizados no desenvolvimento.

SPRING Data JPA Documentation. Disponível em: <https://spring.io/projects/spring-data-jpa>. Acesso em: 25 nov. 2024. Descrição: Documentação oficial sobre o Spring Data JPA, explicando como integrar repositórios com bancos de dados relacionais.

GITHUB. Exemplos de projetos Spring Boot. Disponível em: <https://github.com/spring-projects/spring-boot>. Acesso em: 25 nov. 2024. Descrição: Exemplos de projetos reais utilizando Spring Boot, com implementações práticas de controladores, repositórios e mapeamento de entidades.

JAVA Persistence API (JPA) Specification. Disponível em: <https://www.jcp.org/en/jsr/detail?id=338>. Acesso em: 25 nov. 2024. Descrição: Especificação oficial do JPA, que explica o mapeamento objeto-relacional usado nas entidades.